

Master AI Engineer

Stéphanie Duhem

Juillet 2026

NOTE TECHNIQUE

PROJET 8

Traitez les images pour le système embarqué d'une voiture autonome

SOMMAIRE

- 1- Contexte du projet
- 2- État de l'art de la segmentation d'images
- 3- Description détaillée des architectures U-Net retenues
- 4- Stratégies d'optimisation
- 5- Synthèse des résultats obtenus
- 6- Conclusion et perspectives
- Annexes

1- Contexte du projet

Future Vision Transport est une entreprise spécialisée dans la conception de systèmes embarqués de vision pour la conduite autonome. L'objectif à long terme est de fournir à une voiture autonome une compréhension fine et en temps réel de son environnement visuel.

Au sein de l'équipe R&D, chaque membre intervient sur un bloc spécifique du système : l'acquisition d'images, le traitement, la segmentation et la prise de décision. Ce découpage modulaire permet une intégration progressive des composants.

Mon rôle porte sur la partie segmentation d'images, qui reçoit les images traitées puis transmet les masques de segmentation au module décisionnel. Ce module est central dans la chaîne de traitement : il doit être suffisamment précis pour garantir la sécurité des décisions aval, et suffisamment rapide pour répondre aux contraintes temps réel du système embarqué.

Plan d'action

Objectifs

L'objectif principal est d'entraîner un modèle de segmentation d'images capable d'identifier les 8 catégories agrégées de l'environnement routier issues du dataset Cityscapes : void, flat, construction, object, nature, sky, human, vehicle.

Ce modèle doit être intégré dans une pipeline entièrement déployée sur le Cloud, composée :

- d'une API de prédiction simple à utiliser, permettant à l'équipe décisionnelle d'obtenir rapidement un masque de segmentation depuis une image brute ;
- d'une application de démonstration (Streamlit) pour visualiser les prédictions de manière interactive.

Nous avons donc 3 grandes étapes avec la modélisation (incluant l'exploration des données et la sélection du meilleur modèle), le déploiement (API + application) et la documentation.

Contraintes techniques

Le projet doit répondre à plusieurs contraintes :

- **Rapidité de la prédiction** : pour un système de vision par ordinateur embarqué, le temps de traitement est critique. Le modèle doit être suffisamment léger pour tourner dans un temps raisonnable.
- **Gestion du déséquilibre des classes** : le dataset Cityscapes présente un fort déséquilibre (la classe flat représente la majorité des pixels, tandis que les classes human et object sont minoritaires mais critiques pour la sécurité).
- **Ressources de calcul limitées** : les entraînements sont réalisés sur GPU distant, avec des contraintes de temps et de mémoire qui conditionnent les choix d'architecture et d'hyperparamètres.
- **Déploiement cloud** : le modèle final doit être déployé sous forme d'une API de prédiction et une application web de visualisation, toutes deux accessibles sur le Cloud.

Absence de fuite d'information

L'absence de fuite d'information entre les jeux de données est garantie structurellement.

Le split train/val est effectué sur les chemins de fichiers (via `train_test_split`) avant tout chargement en mémoire. Le **générateur ImageSegmentationDataset** ne reçoit que la liste d'indices qui lui appartient : il est impossible qu'une image de validation soit vue pendant l'entraînement. Le jeu de test (images Cityscapes val officielles, séparées du train dès la création du dataset) n'est chargé qu'une seule fois, après la sélection du modèle final, pour le calcul des métriques de test reportées dans le tableau comparatif. Il n'y a aucun chevauchement entre les trois splits (train, val, test).

2- État de l'art de la segmentation d'images

Définitions

La segmentation d'images est une tâche clé en vision par ordinateur, particulièrement pour les véhicules autonomes. Elle consiste à associer une classe ou un label à chaque pixel d'une image.

On distingue plusieurs niveaux de granularité :

- **Segmentation sémantique** : chaque pixel est associé à une classe (ex : route, véhicule, piéton), sans distinction des instances.
- **Segmentation d'instance** : chaque objet individuel est identifié séparément (deux piétons = deux instances distinctes).
- **Segmentation panoptique** : combinaison des deux, distinguant les objets dénombrables en instances des zones continues sémantiques.

Usages clés

La segmentation d'image est centrale dans de nombreux domaines : la conduite autonome (routes, piétons, obstacles), l'imagerie médicale (organes, lésions), l'imagerie satellitaire (occupation des sols, agriculture), la robotique industrielle (manipulation d'objets) et la réalité augmentée (segmentation de fond).

Fondations et évolutions : les CNN

La segmentation d'images moderne repose principalement sur le Deep Learning avec des architectures CNN :

- U-Net (2015) : une structure en « U » avec des skip connections, une excellente localisation spatiale, conçue pour la segmentation médicale avec peu de données annotées.
- SegNet (2015) : avec un encodeur VGG16 qui garde uniquement les indices de pooling, léger en utilisation de mémoire.
- FPN (2017) : une pyramide multi-échelle top-down avec connexions latérales, elle capture les objets à différentes échelles.
- DeepLabV3+ (2018) : module ASPP (Atrous Spatial Pyramid Pooling) combinant plusieurs taux de dilatation, il était le meilleur sur Cityscapes à sa sortie.

Vers l'efficacité et les foundation models

Actuellement la segmentation évolue vers : l'usage de backbones pré-entraînés (ResNet, ViT, encodeur CLIP) réutilisés comme encodeurs dans des architectures encodeur-décodeur ; les Transformers (ViT, Swin Transformer) pour la modélisation de dépendances longues portées ; et les foundation models pré-entraînés sur des corpus massifs et polyvalents.

L'objectif actuel est de réduire le besoin en données annotées, vers le zero-shot et few-shot learning (prédire des classes jamais ou peu vues explicitement lors de l'entraînement), via des modèles guidés par prompts.

- SAM 2 (2024) : foundation model unifié image+vidéo (Transformer + prompt encoder + mask decoder), excellent en segmentation interactive mais peu adapté à une classification sémantique directe sans adaptation.
- OMG-Seg (CVPR 2024) : framework multitâche unifié (6 tâches de segmentation) s'appuyant sur un backbone CLIP gelé avec des décodeurs spécifiques par tâche — démontre la faisabilité d'un modèle universel de segmentation.

Ces modèles confirment un basculement vers des architectures unifiées et guidées par prompts. Mais ils restent particulièrement coûteux en calcul et ne répondent pas directement aux contraintes embarquées temps réel de notre projet.

Choix pour ce projet

Pour notre projet, nous utilisons des architectures U-Net comme première approche, progressivement plus complexes. D'une version légère from scratch en guise de baseline, nous irons vers des architectures plus complexes avec backbone pré-entraîné en mode transfer learning. Puis nous comparerons les meilleurs de ce benchmark avec leur version fin-tuned, avec ou sans augmentation des données. Ce choix est cohérent avec les contraintes embarquées et la disponibilité limitée de données annotées.

3- Description détaillée des architectures U-Net retenues

Présentation générique U-Net

UNet est un réseau de neurones convolutionnel conçu pour la segmentation d'images, initialement en bioimagerie, puis généralisé à toutes les tâches de segmentation.

Il s'agit d'une architecture encodeur-décodeur symétrique avec des **skip connections** (concaténation des feature maps de l'encodeur vers le décodeur). Ces connexions permettent de combiner l'information sémantique globale et l'information spatiale locale.

- **Encodeur** : compresse et enrichit sémantiquement (convolutions + downsampling).
- **Décodeur** : reconstruit la résolution spatiale en réinjectant des détails fins via les skips.
- **Avantages** : On a une localisation précise des frontières, robustesse contextuelle, efficace avec données annotées limitées. Ainsi que la flexibilité : on peut substituer l'encodeur avec un backbone pré-entraîné.

Architectures retenues : du plus simple au plus complexe

Conformément aux bonnes pratiques, les architectures ont été testées du plus simple au plus complexe, en établissant une baseline solide avant d'introduire le transfer learning :

- UNet-mini (~0,19M paramètres) — modèle de référence (baseline), entraîné from scratch
- MobileNetV3Small-UNet (~1,5-2M paramètres) — backbone léger pré-entraîné ImageNet, candidat naturel
- VGG16-UNet (~15-20M paramètres) — backbone classique pré-entraîné ImageNet, challenger
- ResNet50-UNet (~25M paramètres) — backbone performant pré-entraîné ImageNet, upper bound

UNet-mini (~0,19 M paramètres)

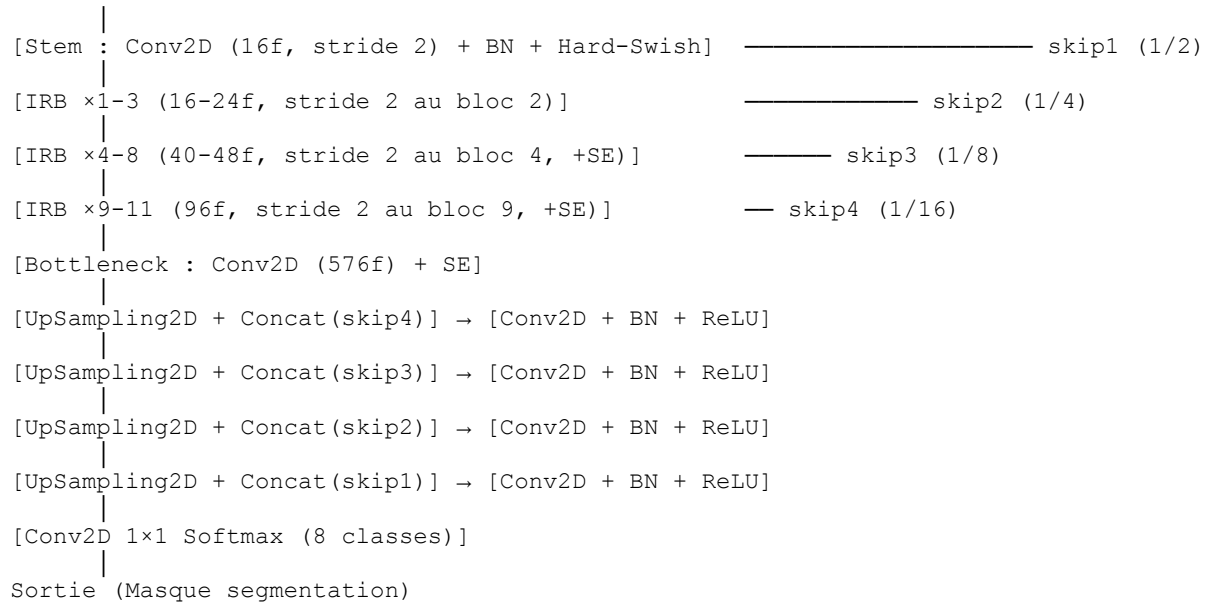
Architecture encodeur-décodeur légère entraînée from scratch sans backbone pré-entraîné. Profondeur réduite à 2 niveaux de downsampling, avec skip connections. Utilisée comme baseline pour établir une référence basse avant l'introduction du transfer learning. Très rapide à entraîner (11,7 min), mais limitée sur les petits objets et les classes minoritaires en raison de la profondeur réduite et de l'absence de normalisation (BatchNorm).

```
Entrée (256×512×3)
|
[Conv2D×2 + ReLU (64f)] ————— skip1
[MaxPool 2×2]
|
[Conv2D×2 + ReLU (128f)] ————— skip2
[MaxPool 2×2]
|
[Bottleneck : Conv2D×2 (256f)]
|
[UpSampling2D + Concat(skip2)] → [Conv2D×2 + ReLU (128f)]
|
[UpSampling2D + Concat(skip1)] → [Conv2D×2 + ReLU (64f)]
|
[Conv2D 1×1 Softmax (8 classes)]
|
Sortie (Masque segmentation)
```

MobileNetV3Small-UNet (~1,5-2 M paramètres)

Backbone MobileNetV3Small pré-entraîné sur ImageNet, encodeur compact optimisé pour les contraintes embarquées. Décodeur avec skip connections multi-échelles, UpSampling bilinéaire + BatchNorm + ReLU. Bon compromis performance/légèreté, adapté à un déploiement edge.

Entrée (256×512×3)



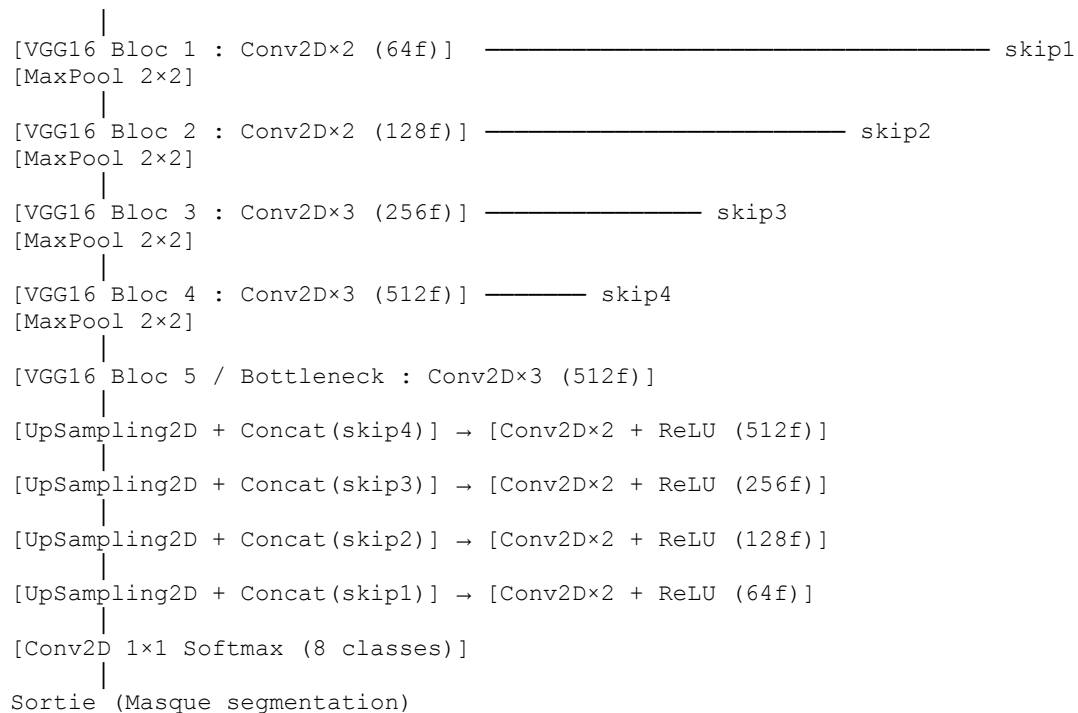
IRB = Inverted Residual Block (conv1×1 expansion → depthwise conv → conv1×1 projection)

SE = Squeeze-and-Excitation · BN = Batch Normalization

VGG16-UNet (~15-20 M paramètres)

Backbone VGG16 pré-entraîné sur ImageNet, architecture classique à convolutions séquentielles 3×3 empilées, 5 niveaux de downsampling. Encodeur lourd et peu adapté aux contraintes embarquées, testé comme variante pour mesurer l'impact d'un backbone plus lourd.

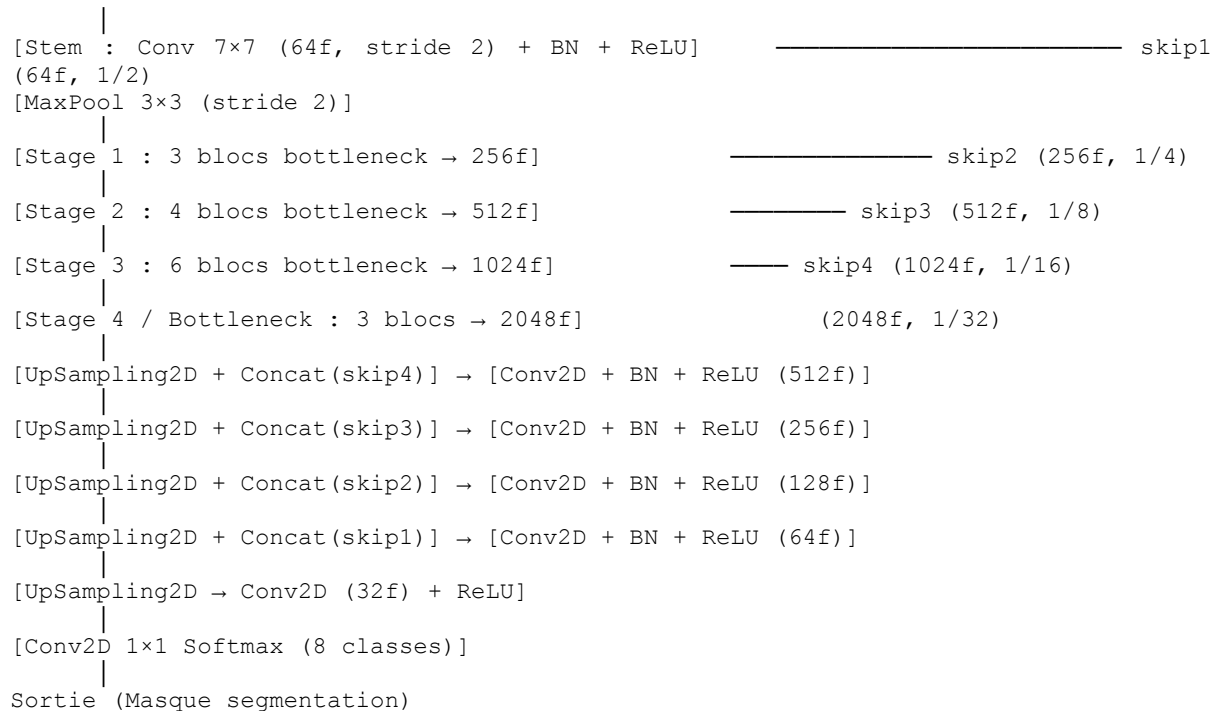
Entrée (256×512×3)



ResNet50-UNet (~25 M+ paramètres, ~314 Mo)

Backbone ResNet50 pré-entraîné sur ImageNet avec blocs résiduels (skip connections internes). Architecture la plus lourde du benchmark, elle est retenue comme upper bound de performance.

Entrée (256×512×3)



Bloc bottleneck ResNet : Conv1×1 → Conv3×3 → Conv1×1 + connexion résiduelle (identity shortcut)

Critère de sélection du modèle final

Le choix final entre les quatre architectures repose sur trois axes pondérés selon le cahier des charges :

- mIoU / Dice sur le jeu de validation — qualité de segmentation (critère principal).
- Taille du modèle et coût d'inférence — critère de poids fort dans un contexte embarqué (le système de décision doit tourner en temps réel sur du matériel contraint).

Tension performance / contrainte embarquée

Le modèle finalement retenu pour le déploiement, ResNet50-UNet, pèse environ **314 Mo**. Cette taille interroge sur la cohérence avec un cas d'usage embarqué : un backbone plus léger comme MobileNetV3Small (≈1,5-2 M de paramètres contre ~25 M+ pour ResNet50) aurait été un choix plus naturel pour répondre à la contrainte de rapidité d'obtention de la prédiction énoncée en introduction. Le choix de ResNet50-UNet se justifie ici comme un **upper bound de performance** pour cette première itération, à reconsidérer pour une mise en production réelle sur matériel embarqué (cf. perspectives, section 6).

Les stratégies d'optimisation communes aux architectures (DiceFocal Loss, pondération des classes, data augmentation, fine-tuning) sont détaillées en section 4.

4- Stratégies d'optimisation

Fonction de perte : DiceFocal Loss

Pour la segmentation d'images multi-classes, il existe plusieurs fonctions de perte :

- Categorical Crossentropy : elle compare la distribution des classes prédites (softmax) à la vérité terrain, elle peut être pondérée pour gérer le déséquilibre. Limite : sensible au déséquilibre.
- Dice Loss : elle mesure le recouvrement entre masque prédit et masque réel, crucial pour la précision des contours et des petits objets (piétons, véhicules lointains). Elle optimise directement le Dice coefficient (similaire au F1-score), moins sensible au déséquilibre, cela favorise le bon recouvrement des classes minoritaires.
- Focal Loss (crossentropy modifiée) : elle accorde plus d'importance aux exemples difficiles mal classés, et du coup réduit l'impact des exemples faciles. Efficace sur les datasets déséquilibrés.
- Combo Loss (Dice + Focal) : elle combine les avantages des deux. Elle est très utilisée en compétition et en recherche pour la segmentation médicale ou urbaine.

Pour notre projet, nous choisissons la combinaison Dice Loss + Focal Loss, contractée en **DiceFocal Loss**, pour pondérer au mieux le déséquilibre des classes, fréquent en milieu urbain par la présence massive de pixels route ou ciel en opposition à la rareté des piétons et des panneaux.

Cette combinaison nous permet donc d'obtenir de meilleurs résultats sur les classes minoritaires et critiques pour la sécurité. Pour des raisons de coût et de temps, les autres fonctions de perte n'ont pas été testées, mais cela constitue une perspective d'amélioration.

Gestion du déséquilibre des classes

Il existe un important déséquilibre entre les classes, ce qui est logique car certains objets sont présents sur l'ensemble des images (routes, bâtiments) tandis que d'autres sont plus rares (piétons, panneaux). Les poids des classes sont utilisés lors de l'entraînement du modèle, dans le calcul de la DiceFocal Loss. Le modèle aura ainsi appris à mieux détecter les classes rares, car une erreur de prédiction sur ces classes sera plus pénalisante. Les poids de classes sont intégrés dès la création du dataset, via la classe ImageSegmentationDataset.

Cette stratégie de pondération s'appuie directement sur l'analyse de la distribution des pixels par catégorie réalisée en phase d'exploration (cf. data.ipynb, section « Déséquilibre des classes »).

Deux méthodes classiques existent pour calculer des poids de classes :

- Inverse Frequency : $w_c = 1 / f_c$, où $f_c = N_pixels_classe / N_pixels_total$.
Plus une classe est rare, plus son poids est élevé.
- Median Frequency Balancing : $w_c = \text{Médiane}(\text{toutes fréquences}) / f_c$.
Atténuation de l'effet des classes extrêmement rares.

Poids retenus (méthode Inverse Frequency, calculés depuis class_pixel_counts dans data.ipynb) :

0-void : 0.048 | 1-flat : 0.013 | 2-construction : 0.022 | 3-object : 0.276

4-nature : 0.032 | 5-sky : 0.137 | 6-human : 0.406 ← classe critique sécurité | 7-vehicle : 0.066

Hyperparamètres optimisés

- Fonction de loss : DiceFocal Loss (combinaison Dice + Focal).
- Pondération des classes : méthode Inverse Frequency, poids allant de 0.013 (flat, classe majoritaire) à 0.406 (human, classe critique).
- Nombre d'epochs : 20 pour le benchmark en transfert learning et 50 pour les phases d'optimisation, contrôlé par Early Stopping.
- Learning rate adaptatif : ReduceLROnPlateau (factor=0.5, patience=3) pour les runs de 50 epochs.
- Stratégie d'optimisation : trois variantes comparées pour MobileNetV3 et ResNet50 — encodeur gelé (transfer learning pur) vs. fine-tuning (dégel complet de l'encodeur) avec ou sans augmentation des données (albugmentation intégré dans le générateur de dataset).

Early stopping et réduction du learning rate

Pour optimiser l'entraînement et éviter le sur-apprentissage, un mécanisme d'early stopping est mis en place : l'entraînement s'arrête automatiquement si la métrique de validation (val_dice) ne s'améliore plus pendant un certain nombre d'epochs (patience=5). Cela permet de sauvegarder le meilleur modèle rencontré pendant l'entraînement, sans risquer de surapprentissage.

En complément, une réduction automatique du learning rate (ReduceLROnPlateau) est appliquée : si la val_loss stagne pendant 3 epochs consécutives, le learning rate est réduit de moitié. Ces deux techniques garantissent une convergence stable et limitent le sur-apprentissage.

5- Synthèse des résultats obtenus

5.1 Métriques d'évaluation retenues

Deux métriques principales ont été retenues pour évaluer et comparer les modèles :

IoU (Intersection over Union) par classe

C'est une métrique standard en conduite autonome (benchmark Cityscapes, KITTI, nuScenes). Elle mesure le ratio entre l'intersection et l'union des zones prédites et réelles pour chaque classe :

$$\text{IoU} = \text{TP} / (\text{TP} + \text{FP} + \text{FN})$$

où TP = pixels correctement classés dans la classe, FP = pixels faussement assignés à la classe, FN = pixels de la classe manqués. Robuste au déséquilibre de classes (ne tient pas compte des vrais négatifs). La **mIoU** (moyenne sur toutes les classes) est la métrique de référence Cityscapes.

Dice coefficient

Complémentaire à l'IoU, mesure le chevauchement entre prédiction et vérité terrain :

$$\text{Dice} = 2 \cdot \text{TP} / (2 \cdot \text{TP} + \text{FP} + \text{FN})$$

Moyenne harmonique de la précision et du rappel, elle est plus sensible aux petits objets et aux classes minoritaires (human, object), qui sont critiques en sécurité routière. C'est également la métrique optimisée directement par la DiceFocal Loss, ce qui assure une cohérence entre l'objectif d'entraînement et l'évaluation.

Pixel accuracy

Calculée en complément mais non retenue comme métrique principale : avec un fort déséquilibre de classes (flat représente la majorité des pixels Cityscapes), un modèle prédisant uniquement flat obtiendrait une pixel accuracy élevée mais une mIoU quasi nulle — cette métrique est trompeuse en présence d'une classe dominante.

5.2 Tableau comparatif des architectures testées

Les résultats suivants sont issus de l'expérience MLflow (experiment_id : 733779452140988414). Toutes les métriques reportées sont calculées sur le jeu de validation (sauf colonne test). La durée d'entraînement est indiquée à titre comparatif.

Modèle	Stratégie	Params	mIoU val	Dice val	mIoU test	Dice test	IoU humain (val)	IoU véhicule (val)	Durée	Inférence/img
UNet-mini	from scratch	~0,2M	0.349	0.430	0.328	0.410	0.011	0.202	11min	0,18 sec
MobileNet V3Small	encodeur gelé	~2 M	0.601	0.639	0.582	0.622	0.328	0.690	2,6 h	0,29 sec
VGG16	encodeur gelé	~15M	0.562	0.604	0.549	0.593	0.385	0.000	2,4 h	0,21 sec
ResNet50	encodeur gelé	~25 M+	0.742	0.768	0.717	0.740	0.610	0.841	3,1 h	0,39 sec
MobileNet V3Small	fine-tuning	~2 M	0.661	0.696	0.637	0.676	0.418	0.749	8,3 h	0,32 sec
ResNet50	fine-tuning	~25 M+	0.762	0.789	0.741	0.770	0.656	0.855	6,8 h	0,38 sec
MobileNet V3Small	fine-t. + augment.	~2 M	0.652	0.691	0.626	0.669	0.385	0.747	10 h	0,30 sec
ResNet50	fine-t. + augmentation	~25 M	0,760	0,787	0.738	0.769	0,639	0,854	12h	0,38 sec

Nos principales observations :

- UNet-mini confirme son rôle de baseline : val_dice 0.430 sert de référence basse pour quantifier le gain apporté par le transfer learning (+0.359 points de Dice en fine-tuning ResNet50).
- VGG16 est écarté : IoU vehicle = 0.000 sur tous les splits indique un problème d'apprentissage de la classe vehicle (anomalie, non déployé).
- MobileNetV3Small-UNet fine-tuned (~17 Mo, val_dice 0.696) est un choix pragmatique pour la démo sur CPU Azure B1 et le candidat naturel pour un déploiement embarqué contraint. MobileNetV3Small offre un bon compromis performance / taille / latence dans ce contexte.
- ResNet50 fine-tuned obtient les meilleures performances globales (val_dice 0.789, val_mIoU 0.762). Mais en raison de nos contraintes, il ne pourra pas être déployé.
- L'impact de l'augmentation de données est soit négligeable sur ResNet50 (une perte minime de -0.002 sur la Dice), soit légèrement négatif pour MobileNetV3 (une perte plus sensible de -0.007 sur la Dice). Les augmentations choisies n'ont pas apporté de diversité supplémentaire significative sur Cityscapes.

5.3 Taille des modèles

Au-delà des métriques de performance, la taille du modèle constitue un second indicateur de comparaison déterminant dans ce contexte embarqué :

Modèle	Taille	Remarque
UNet-mini	~2 Mo	Très léger, from scratch, peu performant
MobileNetV3Small-UNet	~10Mo	Modèle au backbone léger, adapté CPU, performance en hausse
VGG16-UNet	~119 Mo	Lourd et anomalie sur vehicle
ResNet50-UNet	~494 Mo	Meilleur modèle sur les métriques mais trop lourd pour embarqué réel

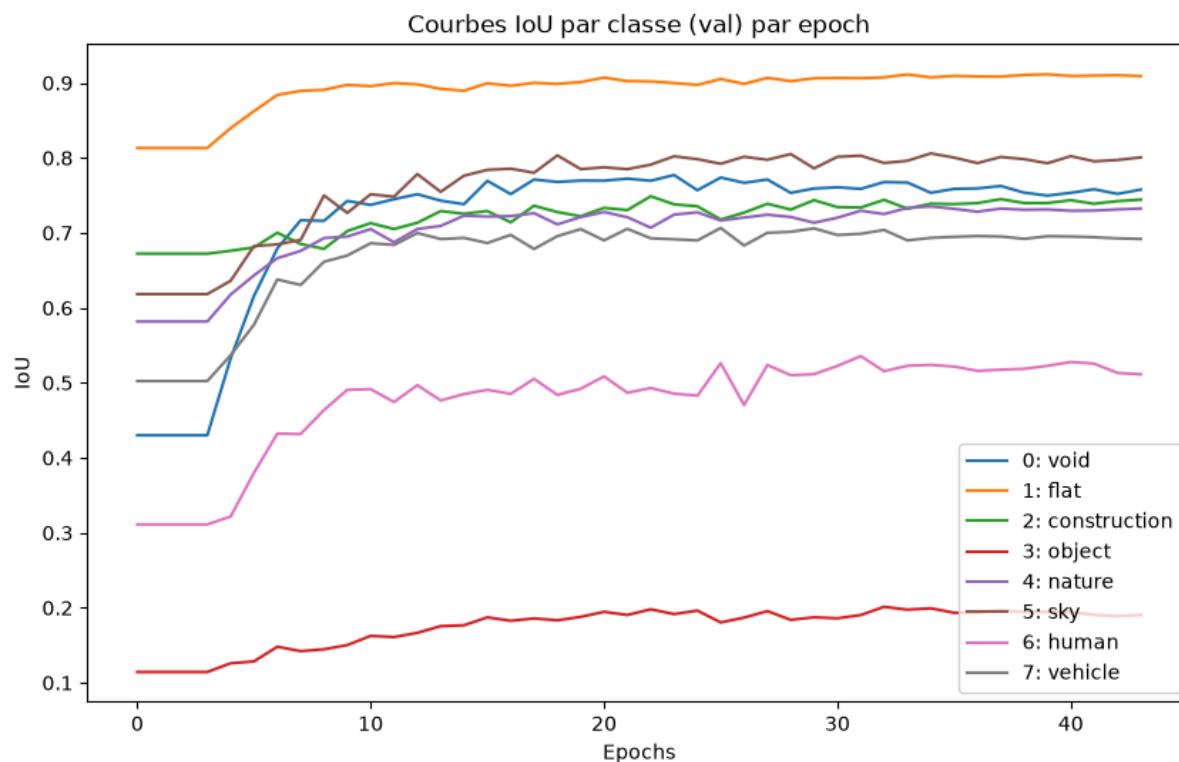
Ce critère est directement lié au cahier des charges justifie le choix du MobileNetV3Small comme une alternative sérieuse à une mise en production réelle.

5.4 Analyse détaillée du modèle retenu : MobileNetV3Small-UNet fine-tuning

Run MLflow retenu : `run_id 3a82bfc48cec4e799f07cc287dc786cc`

Split	Dice	mIoU	Pixel Accuracy
Train	0.776	0.72	0.908
Val	0.696	0.661	0.877
Test	0.676	0.637	0.859

L'écart train/val (Dice : 0.776 vs 0.696, soit -0.080) indique un overfitting modéré et contenu. Cet écart reste néanmoins inférieur à celui observé sur ResNet50 (-0.115), ce qui confirme que le modèle plus léger généralise mieux sur ce volume de données (2 975 images). Les mécanismes mis en place (Early Stopping, ReduceLROnPlateau, Class Weights) ont permis de contenir cet overfitting.

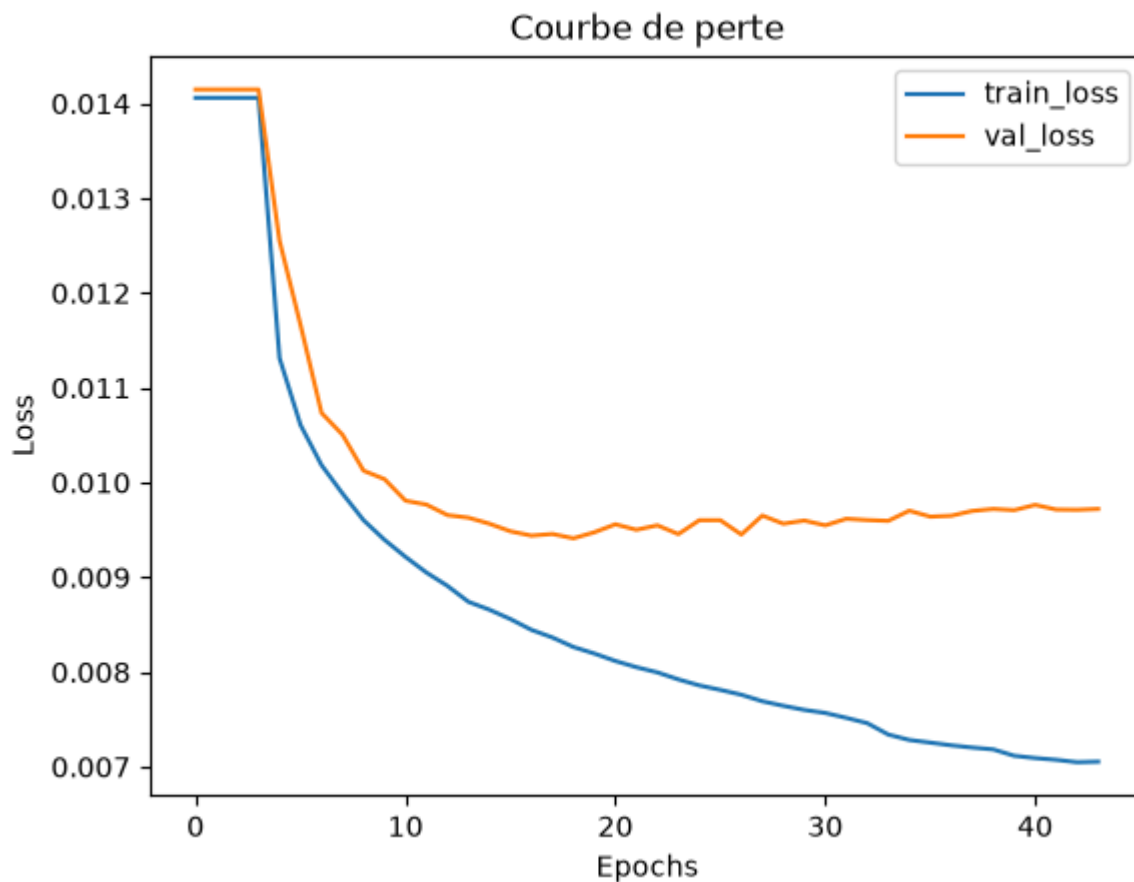


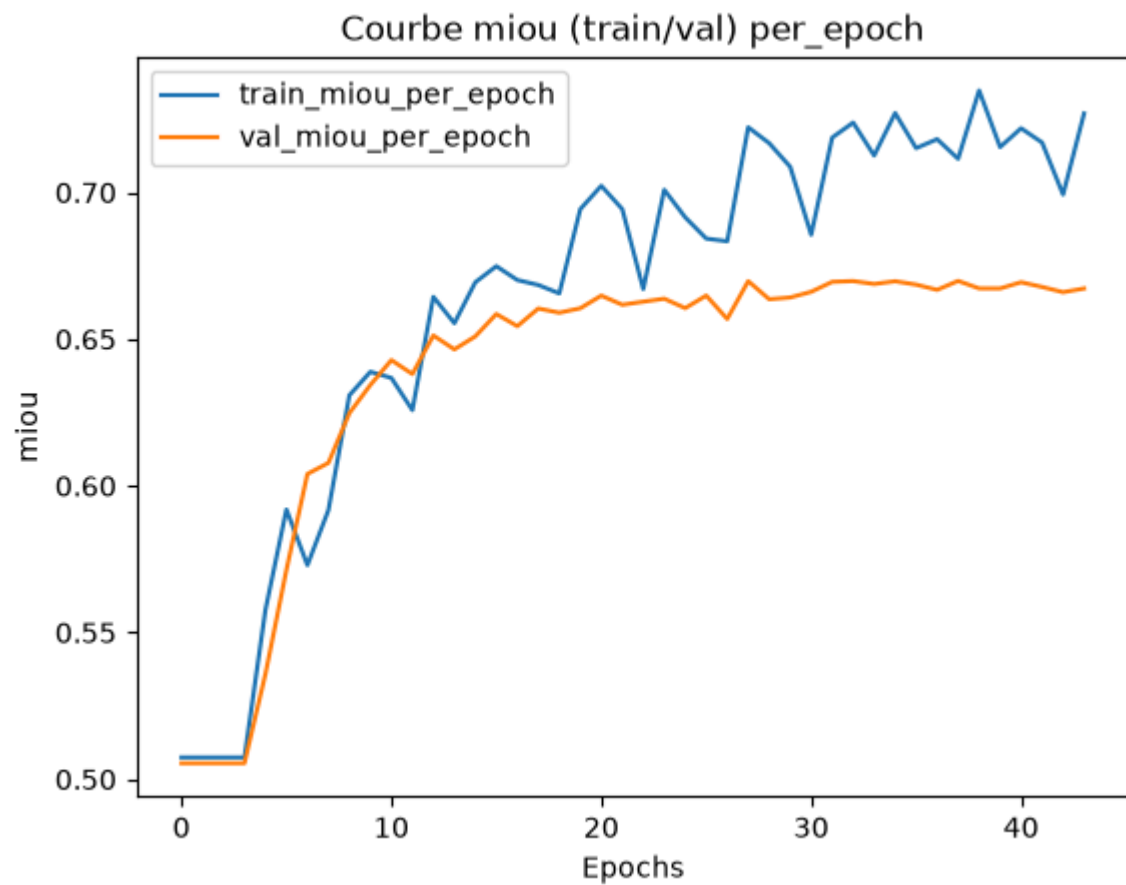
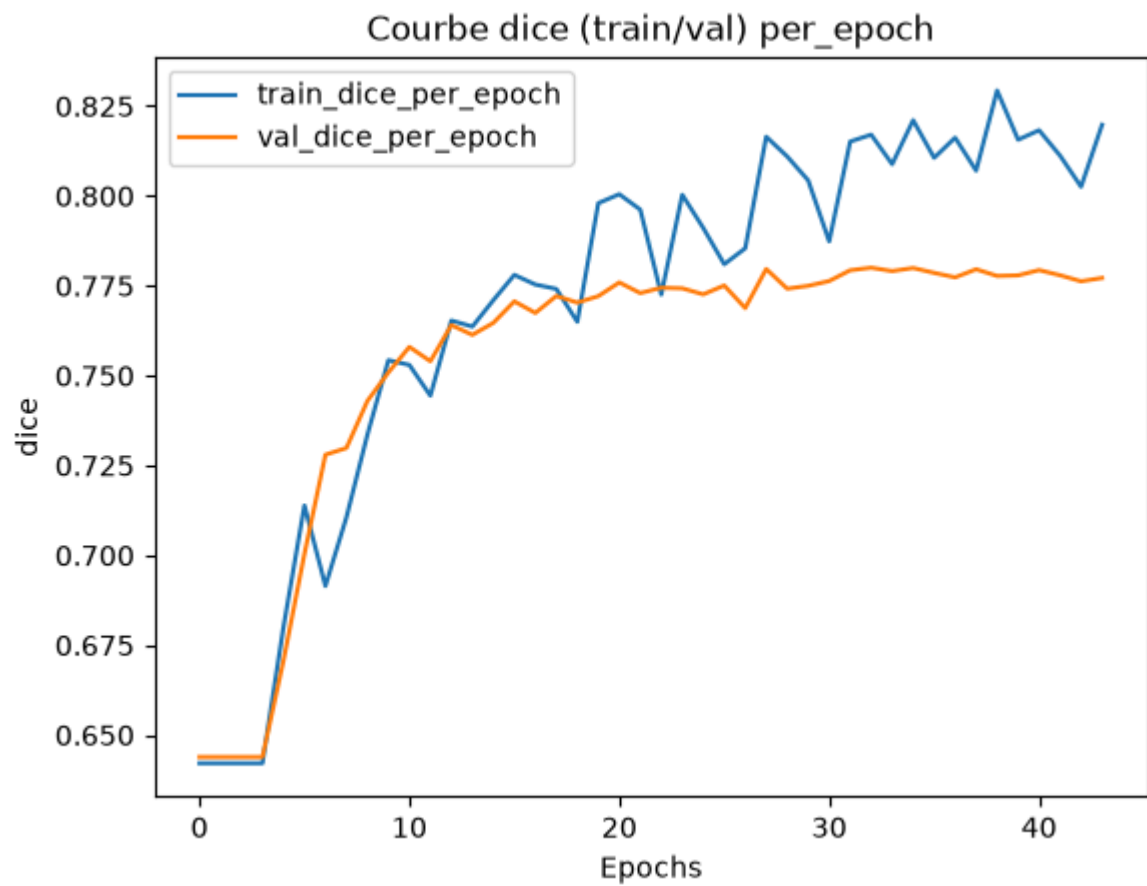
Classes difficiles et critiques pour la sécurité :

- IoU human (val) : 0.418, c'est la classe minoritaire critique (piétons, cyclistes). Son résultat est inférieur à celui du ResNet50 (0.656) mais cohérent avec la capacité réduite du backbone (2M contre 25 M+ paramètres).
- IoU vehicle (val) : 0.749, pour cette classe on observe donc une bonne segmentation.

5.5 Analyse de l'entraînement

- L'Early stopping a été déclenché à l'epoch 47 (patience=10 à partir de l'epoch 37).
- Le ReduceLROnPlateau s'est déclenché en cours d'entraînement (factor=0.5, patience=5). Cette convergence est plus progressive qu'un backbone lourd, ce comportement est attendu avec un encodeur MobileNetV3Small.





5.6 Impact de l'optimisation des modèles : augmentation de données

Comparaison des résultats avec ou sans augmentation sur MobileNetV3Small, 50 epochs :

Configuration fine-tuning	Val Dice	Val mIoU	IoU human (val)	Train Dice	Durée
Sans augmentation	0.696	0.661	0.418	0.776	8,3 h
Avec augmentation	0.691	0.652	0.385	0.515	10,1 h

Techniques d'augmentation appliquées :

- Flip horizontal : en symétrie, cohérent avec la géométrie des scènes routières)
- Variation de luminosité et contraste : robustesse aux conditions d'éclairage variables
- Légère distorsion géométrique : robustesse aux variations de caméra embarquée

Analyse

L'augmentation produit des résultats quasi identiques à la configuration sans augmentation sur MobileNetV3Small avec un $\Delta \text{val_dice} = -0.005$, et un $\Delta \text{val_mIoU} = -0.009$. Ce résultat mérite une lecture nuancée, car il ne signifie pas que l'augmentation est inefficace.

L'indicateur clé est que le train dice à 0.515 est très inférieur au val dice à 0.691. C'est un comportement inverse de l'overfitting, typique de l'augmentation forte : les batches d'entraînement sont rendus artificiellement plus difficiles, ce qui empêche le modèle de mémoriser les exemples de train. En conséquence, la courbe de train dice reste basse pendant tout l'entraînement, tandis que le modèle généralise correctement sur le val (non augmenté). Ce phénomène est documenté dans la littérature (Shorten & Khoshgoftaar, 2019 — A survey on Image Data Augmentation for Deep Learning) et constitue un signal de bonne généralisation, pas d'un entraînement défaillant.

Pourquoi la val_dice ne s'améliore pas davantage ?

Nous avons deux hypothèses :

- MobileNetV3Small est peut-être déjà au plateau de sa capacité avec 50 epochs et l'augmentation nécessite généralement plus d'epochs pour produire un gain visible.
- Les augmentations appliquées sont adaptées mais peut-être insuffisamment agressives pour ce dataset qui présente déjà une bonne diversité de situation en milieu urbain allemand.

6- Conclusion et perspectives

Déploiement

Le modèle ResNet50-UNet (fine-tuning) a été déployé sous forme d'API Flask indépendante :

- Entrée : image sérialisée (base64)
- Sortie : masque de segmentation sérialisé (8 classes visibles, encodé en base64)

Le déploiement est effectué manuellement via Azure CLI :

- API Flask (repo P8_api_app) : image Docker buildée vers Azure Container Registry (acrprojet8) via `az acr build`, puis démarrage de l'App Service (`az webapp restart`).
- Frontend Streamlit (repo P8_front_app) : déployé via `zip deploy` (`az webapp deploy --type zip`).
- Un pipeline CI/CD automatisé (GitHub Actions) constitue une perspective d'amélioration identifiée — le déploiement est aujourd'hui entièrement manuel.
- URL API : <https://projet8-api-e8a2apghhehekc4.swedencentral-01.azurewebsites.net>
- URL Application : <https://projet8-front.azurewebsites.net>

La chaîne de connexion Azure Storage (AZURE_STORAGE_CONNECTION_STRING) est stockée en variable d'environnement de l'App Service Azure, non exposée dans le code source.

Perspectives

Sur le plan des données

- Élargir le jeu d'entraînement à des conditions météo variées (Foggy Cityscapes, Rain Cityscapes) pour améliorer la robustesse en conditions réelles.
- Tester d'autres méthodes de pondération (Median Frequency Balancing) et comparer empiriquement à l'Inverse Frequency retenue.

Sur le plan de l'optimisation des modèles

- Tester un dégel progressif de l'encodeur (par couches) plutôt qu'un gel total, pour évaluer le gain sur les classes minoritaires.
- Comparer ResNet50 avec et sans augmentation (non réalisé par contrainte de calcul).
- Réévaluer le compromis poids/performance : MobileNetV3Small bien optimisé pourrait constituer un bien meilleur candidat pour un déploiement embarqué réel.
- Étudier la quantification (TFLite) pour réduire la taille du modèle (pour REsent de 494Mo à 50 Mo).

Sur le plan applicatif

- Tester la segmentation d'instance pour distinguer les objets individuels (compter les piétons), utile au système de décision aval. Tester également une segmentation panoptique (combinaison de sémantique et d'instance)
- Évaluer la latence de l'API en conditions réelles (temps de réponse sur requêtes concurrentes).

Limites rencontrées

- Les contraintes de calcul ne nous ont pas permis de multiplier les tests sur les hyperparamètres (pas de grid search).

- Le choix du jeu de validation du dataset Cityscapes comme jeu de test final, car le jeu de test officiel n'est pas annoté, est une adaptation méthodologique réussie, assumée et signalée.
- L'IoU vehicle à 0.000 pour le VGG16 signale une anomalie que nous n'avons pas résolu (bug potentiel preprocessing VGG16).
- La taille du ResNet50 est inadaptée à un embarqué strict. Le modèle déployé pour la démo, MobileNetV3Small-UNet, offre un bien meilleur compromis taille/latence sur CPU Azure B1.

Annexes

Liens et ressources

- Dépôt GitHub principal (segmentation) : https://github.com/Oxelie/P8_Segmentation_Images
- Dépôt API : https://github.com/Oxelie/P8_api_app
- URL API déployée :
<https://projet8-api-e8a2apghhekc4.swedencentral-01.azurewebsites.net>
- URL Application Streamlit : <https://projet8-front.azurewebsites.net>
- Expérience MLflow (experiment_id) : 733779452140988414

Dataset Cityscapes — Mapping des classes

ID	Catégorie	Exemples de labels originaux (34→8)
0	void	unlabeled, ego vehicle, rectification border, out of roi, static, dynamic, ground
1	flat	road, sidewalk, parking, rail track
2	construction	building, wall, fence, guard rail, bridge, tunnel
3	object	pole, polegroup, traffic light, traffic sign
4	nature	vegetation, terrain
5	sky	sky
6	human	person, rider
7	vehicle	car, truck, bus, caravan, trailer, train, motorcycle, bicycle